

Распознавание мошеннических разговоров

Рудоманова Мария Алексеевна

2 курс, ФРКТ

апрель 2026 г.

GitHub repository:

<https://github.com/rudik-m/sber-ML>

1. Постановка задачи

Требуется разработать алгоритм бинарной классификации телефонных разговоров по аудиофайлам формата `.wav`. Необходимо определить, является ли разговор мошенническим.

Используемое обозначение классов:

- 0 — разговор мошеннический;
- 1 — разговор не мошеннический.

Дополнительные ограничения задачи:

- решение должно быть реализовано на языке Python;
- запрещено использование платных API и библиотек;
- алгоритм должен работать по папке с `.wav`-файлами;
- итог должен возвращаться в виде CSV-файла формата `filename,label`;
- время обработки одной записи не должно превышать 3 минуты.

2. Идея решения

В данной задаче основной информативный сигнал содержится не столько в акустических особенностях речи, сколько в содержании разговора: упоминании кодов из SMS, переводов денег, ссылок на службы безопасности, срочности, кредитов и других характерных для вишинга шаблонов.

Поэтому решение построено вокруг следующего пайплайна:

1. преобразование аудио в текст с помощью ASR;
2. нормализация текста;
3. извлечение текстовых и аудио-признаков;
4. обучение нескольких моделей классификации;
5. построение итогового ансамбля;
6. получение предсказаний по входной папке с файлами.

Здесь ASR (*Automatic Speech Recognition*) — это автоматическое преобразование речи в текст. В данной работе ASR используется как первый этап обработки, после которого алгоритм анализирует уже не сам аудиосигнал, а полученную текстовую расшифровку разговора.

3. Архитектура алгоритма

Пайплайн решения имеет вид:

`wav` → `transcript` → `normalize` → `features` → `model` → `predictions.csv`

Более подробно этапы работы таковы:

1. Для каждого `.wav`-файла выполняется автоматическая транскрибация речи в текст.
2. Полученный текст нормализуется: приводится к нижнему регистру, очищается от лишних пробелов и типичных артефактов ASR.
3. По нормализованному тексту строятся:
 - словарные `trigger`-признаки;
 - `regex-pattern` признаки;
 - текстовые мета-признаки;
 - простые статистические аудио-признаки.
4. На этих признаках обучаются модели:
 - `LogisticRegression`;
 - `CatBoost`;
 - ансамбль вероятностей.
5. Для новой папки с аудиофайлами строятся предсказания и сохраняются в `predictions.csv`.

4. Используемые методы и библиотеки

В решении используются только открытые локальные библиотеки:

- `faster-whisper` — для ASR;
- `scikit-learn` — для `LogisticRegression`, TF-IDF и метрик;
- `catboost` — для табличной модели;
- `pandas`, `numpy`, `scipy` — для обработки данных;
- `librosa`, `soundfile` — для извлечения аудио-статистик;
- `click` — для CLI.

Платные API и внешние облачные сервисы не используются.

5. Признаки

В проекте используются четыре основные группы признаков.

5.1. Trigger features

Это словарные признаки, основанные на заранее собранных триггерных фразах, характерных для мошеннических звонков:

- служба безопасности;
- центральный банк;
- безопасный счёт;
- код из SMS;
- оставайтесь на линии;

- заявка на кредит;
- сим-карта;
- срочно нужны деньги.

Идея этих признаков состоит в том, что телефонное мошенничество часто использует довольно устойчивые речевые шаблоны. Поэтому наличие таких фраз повышает вероятность того, что разговор является мошенническим.

5.2. Pattern features

Используются регулярные выражения для более гибкого поиска шаблонов в тексте, например:

- конструкции про SMS-код;
- конструкции про перевод денег;
- упоминания безопасного счёта;
- ссылки на официальные органы;
- признаки срочности.

Регулярные выражения нужны потому, что после ASR одна и та же фраза может быть распознана немного по-разному. Такой подход позволяет находить не только точные совпадения, но и близкие текстовые варианты.

5.3. Текстовые мета-признаки

Для транскрипта дополнительно рассчитываются:

- длина текста;
- число слов;
- число уникальных слов;
- средняя длина слова;
- число цифр;
- число вопросительных и восклицательных знаков;
- коэффициент повторяемости.

Эти признаки описывают общую структуру и стиль речи. Например, большое число повторов или цифр может быть косвенным сигналом подозрительного разговора.

5.4. Аудио-признаки

Аудио используется как дополнительный, а не основной сигнал. Рассчитываются:

- длительность записи;
- RMS-энергия;
- zero crossing rate;
- spectral centroid;
- spectral bandwidth;
- отношение тишины и речи;
- средняя длина пауз;
- число пауз;
- приблизительная скорость речи.

Эти признаки не пытаются “понять” смысл разговора, но дают модели статистический контекст: насколько запись длинная, как распределены паузы, насколько активна речь и так далее.

6. Используемые модели

В решении рассматриваются три варианта модели.

6.1. LogisticRegression

LogisticRegression — это линейная модель бинарной классификации. Она хорошо подходит для текстовых признаков, особенно для TF-IDF-представления, и удобна тем, что позволяет интерпретировать вклад отдельных признаков.

6.2. CatBoost

CatBoost — это модель градиентного бустинга над решающими деревьями. Она лучше улавливает нелинейные зависимости между признаками и хорошо работает на табличных dense-признаках.

6.3. Ансамбль

Ансамбль — это объединение предсказаний нескольких моделей. В данной работе используется взвешенное усреднение вероятностей:

$$p = 0.7 \cdot p_{\text{logreg}} + 0.3 \cdot p_{\text{catboost}}$$

Такой подход позволяет объединить сильные стороны двух моделей:

- **LogisticRegression** хорошо работает с текстовыми sparse-признаками;
- **CatBoost** лучше использует компактные плотные признаки.

7. Обучение и валидация

Для обучения используются только записи из выборки **samples**. Папка **test** в обучении не участвует и используется только для инференса.

Так как в датасете присутствуют серии похожих файлов, валидация строится не по отдельным записям, а по группам. Группа извлекается из имени файла, например:

- **out_a_13.wav** → группа **out_a**;
- **Nout_b_33.wav** → группа **nout_b**.

Это позволяет избежать утечки информации между **train** и **validation**.

Обычная случайная разбивка по отдельным файлам могла бы дать завышенную оценку качества. Например, если два очень похожих разговора из одной серии попали бы одновременно и в **train**, и в **validation**, модель фактически увидела бы почти тот же пример заранее. Поэтому используется именно **group-based validation**: все файлы одной серии попадают только в одну часть выборки.

В текущем запуске после фильтрации по **split=samples** для обучения использовалось 40 размеченных записей, из которых 34 попали в **train** и 6 — в **validation**.

8. Метрики качества

Для оценки качества используются следующие метрики:

- **Accuracy** — доля правильных ответов среди всех объектов;
- **Precision** — доля действительно мошеннических разговоров среди тех, которые модель пометила как мошеннические;
- **Recall** — доля найденных мошеннических разговоров среди всех мошеннических разговоров;
- **F1-score** — гармоническое среднее между Precision и Recall;
- **ROC-AUC** — интегральная метрика качества ранжирования вероятностей.

Формула F1-score имеет вид:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Для задачи обнаружения мошеннических звонков особенно важен высокий Recall, так как пропуск мошеннического разговора обычно опаснее, чем ложное срабатывание на обычный разговор.

9. Сравнение моделей

В решении рассматривались три варианта:

- `LogisticRegression`;
- `CatBoost`;
- ансамбль вероятностей $0.7 * \text{logreg} + 0.3 * \text{catboost}$.

Актуальные метрики на group-based validation приведены в таблице.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
LogisticRegression	1.0000	1.0000	1.0000	1.0000	1.0000
CatBoost	0.5000	0.2500	1.0000	0.4000	1.0000
Ensemble	1.0000	1.0000	1.0000	1.0000	1.0000

Таблица 1: Метрики моделей на validation-части выборки `samples`

По текущим экспериментам:

- лучшие значения Accuracy, Precision, Recall и F1 показали `LogisticRegression` и ансамбль;
- `CatBoost` показал более слабый результат по Accuracy и F1 на текущем validation-разбиении;
- в качестве итогового варианта для инференса используется ансамбль, поскольку он объединяет преимущества линейной текстовой модели и табличной модели на dense-признаках.

Следует отметить, что обучающая выборка невелика, а validation-часть содержит всего 6 записей. Поэтому полученные метрики нужно интерпретировать осторожно: они отражают качество на текущем учебном наборе, но не являются окончательной оценкой на большой скрытой выборке.

10. Анализ ошибок

Для итоговой модели **ensemble** на validation-части были получены следующие результаты:

- всего validation-записей: 6;
- False Positive: 0;
- False Negative: 0.

То есть на текущем validation-разбиении модель не допустила ошибок классификации. Однако из-за малого размера validation-части этот результат не следует трактовать как окончательную оценку качества на большой независимой выборке.

11. Проверка ограничения по времени

Согласно условию задачи, обработка одной записи не должна превышать 3 минуты.

По фактическим замерам в рамках проведённых запусков:

- транскрибация 100 файлов заняла около 5 минут 29 секунд;
- среднее время транскрибации составило около 3.3 секунды на запись;
- построение признаков 100 файлов заняло около 40 секунд, то есть около 0.4 секунды на запись;
- инференс по 60 тестовым файлам занял около 14 секунд;
- полный путь для новой записи с ASR укладывается примерно в 5–7 секунд на запись.

Следовательно, ограничение по времени выполняется с большим запасом.

12. Формат итогового результата

Итог работы алгоритма сохраняется в CSV-файл формата:

```
filename,label
```

Пример:

```
filename,label
out_d_19.wav,0
Nout_b_32.wav,1
```

Также дополнительно сохраняется расширенный debug-файл с вероятностями и сработавшими триггерами.

13. Как запустить проект

Основной сценарий запуска пайплайна выглядит так:

```
uv sync
uv run vishing inspect --input-dir ./vishing
uv run vishing transcribe --input-dir ./vishing
uv run vishing build-features --input-dir ./vishing
```

```
uv run vishing train-all --features ./vishing/artifacts/features/features.csv
uv run vishing predict --input-dir ./vishing/test --model ensemble
uv run vishing report --input-dir ./vishing
```

После выполнения этих команд итоговые предсказания сохраняются в файл:

```
vishing/artifacts/predictions/predictions.csv
```

14. Достоинства и ограничения решения

Преимущества предложенного подхода:

- решение полностью локальное и воспроизводимое;
- не используются платные API;
- итоговый пайплайн интерпретируемый;
- поддерживается запуск по папке с файлами;
- соблюдается ограничение по времени.

Ограничения:

- качество классификации сильно зависит от качества ASR;
- датасет невелик, поэтому метрики нестабильны;
- аудио используется только как дополнительный источник признаков;
- не используются тяжёлые end-to-end speech-модели.

15. Вывод

В рамках работы был реализован алгоритм распознавания мошеннических разговоров по аудиозаписям. Решение построено как полный воспроизводимый пайплайн: от транскрибации речи до формирования итогового CSV-файла с предсказаниями.

Алгоритм удовлетворяет основным требованиям задачи:

- реализован на Python;
- не использует платные API;
- работает по папке с аудиофайлами;
- возвращает результат в требуемом CSV-формате;
- укладывается в ограничение по времени обработки записи.

С практической точки зрения выбранный подход хорошо подходит для небольшого датасета и даёт интерпретируемое решение, которое можно дальше улучшать за счёт расширения выборки, улучшения ASR и добавления более сложных признаков.